

TP1 : Initiation à Scilab

EXERCICE 1 :

Manipulation des vecteurs et des matrices sur scilab

```
TP1.sce
1 //creation des vecteurs ligne et colonne
2 U=[1, -2, -3, -4]
3 V=[1;3;5]
4 //creation d'une matrice
5 A=[2 -4 4; 2 0 1; 4 1 1]
6 //affichage
7 disp(U);
8 disp(V);
9 disp(A);
10 //les operateurs sur les vecteur
11 B=V*U;
12 C=A*B;
13 //la transposé de B et U
14 disp(B');
15 disp(U');
16 //inverse d'une matrice
17 D=inv(A);
18 //ligne 2 de la matrice
19 disp(C(2,:));
20 //colonne 3 de la matrice C
21 disp(C(:,3));
22 //solution du systeme Ax=V
23 x=A\V;
24 disp(x);
25
```

1. 2. 3. 4.

1.

3.

5.

2. -4. 4.

2. 0. 1.

4. 1. 1.

1. 3. 5.

2. 6. 10.

3. 9. 15.

4. 12. 20.

1.

2.

3.

4.

7. 14. 21. 28.

30.

21.

36.

-1.5

5.

6.

Exercice 2

Définition de f :

```
1 function[y]=f(x)
2 ...y=sin(x)/(x^2+1);
3 endfunction
4 disp(f(0));
5 disp(f(2));
6 disp(f(10^-3));
7 disp(f(10^-13));
8 x=linspace(0,2*3.14,100);
9 plot(x,f,"r");
10
```

Affichage des images

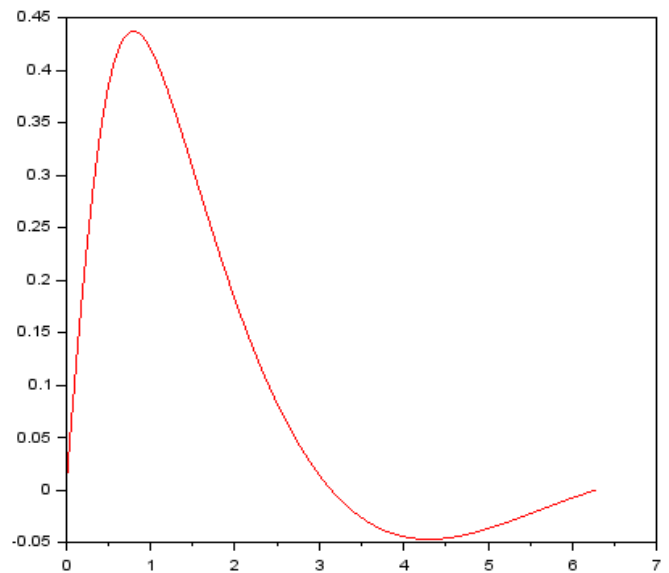
0.

0.1818595

0.0010000

1.000D-13

Graphe de la fonction f (a l'aide de la fonction plot) ;



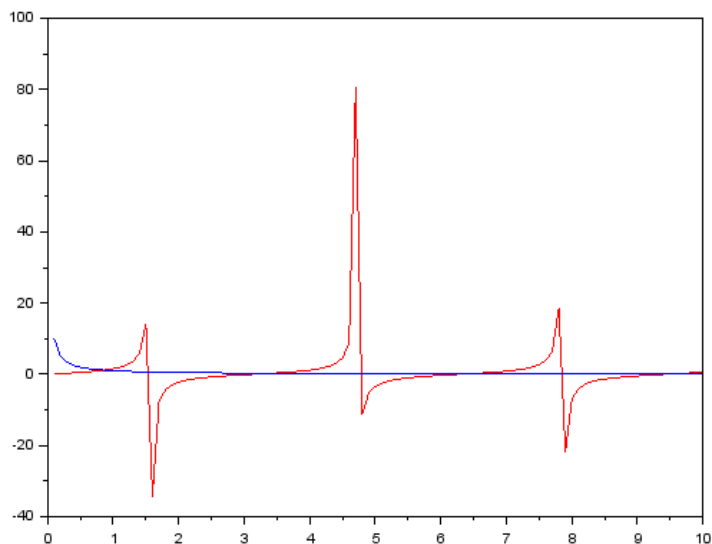
TP2 : Solutions numérique de $f(x) = 0$

Exercice 1 :

Les deux fonctions : $\tan(x)$ et $\frac{1}{x}$

```
function [y]=g(x)
    y=tan(x);
endfunction
function [z]=h(x)
    z=1/x;
endfunction
x=linspace(0.01,10,100);
plot(x,g,"r",x,h,"b");
```

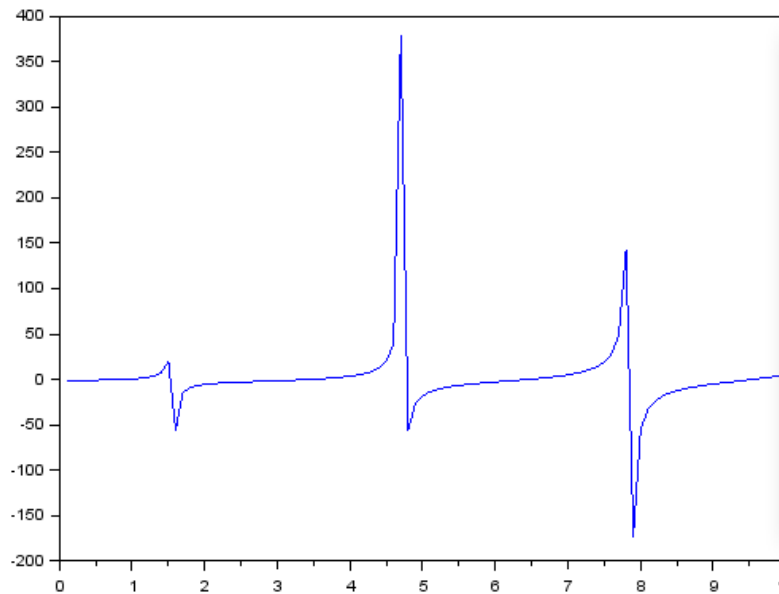
Les courbes des deux fonctions : $\tan(x)$ et $\frac{1}{x}$



D'après la courbe L'encadrement des trois premières solutions strictement positives de l'équation $\tan(x) = \frac{1}{x}$ est : $[0.5, 1.5]$ $[2, 4]$ $[6, 7]$

Programmation d'une fonction Scilab $y = f(x)$ calculant la valeur $f(x)$ pour $f(x) = x \tan(x) - 1$:

```
function [y]=f(x)
    y=x*tan(x)-1;
endfunction
x=linspace(0.01,10,100);
plot(x,f,"b");
```



$$\tan(x) = \frac{1}{x} \text{ alors } x \tan(x) - 1 = 0$$

1. Programmation d'une fonction Scilab ***sol = dichotomie(a, b)*** calculant la dichotomie à une solution comprise entre ***a*** et ***b*** de ***f(x) = 0*** avec 100 iteration:

```
function [sol]=dicho(a, b)
    x=(a+b)/2;
    i=1;
    while(i<100)
        if(f(a)*f(x)<0)
            b=x;
        else
            a=x;
        end
        x=(a+b)/2;
        i=i+1;
    end
    sol=x;
endfunction
sol=dicho(0.5,1.5,10^-5);
disp(sol);
disp(f(sol));
sol=dicho(2,4,10^-5);
disp(sol);
disp(f(sol));
sol=dicho(6,7,10^-5);
disp(sol);
disp(f(sol));
```

Les 3 solutions de f dans les 3 intervalles sont

[0.5 , 1.5] **0.8603336**

[2, 4] **3.4256185**

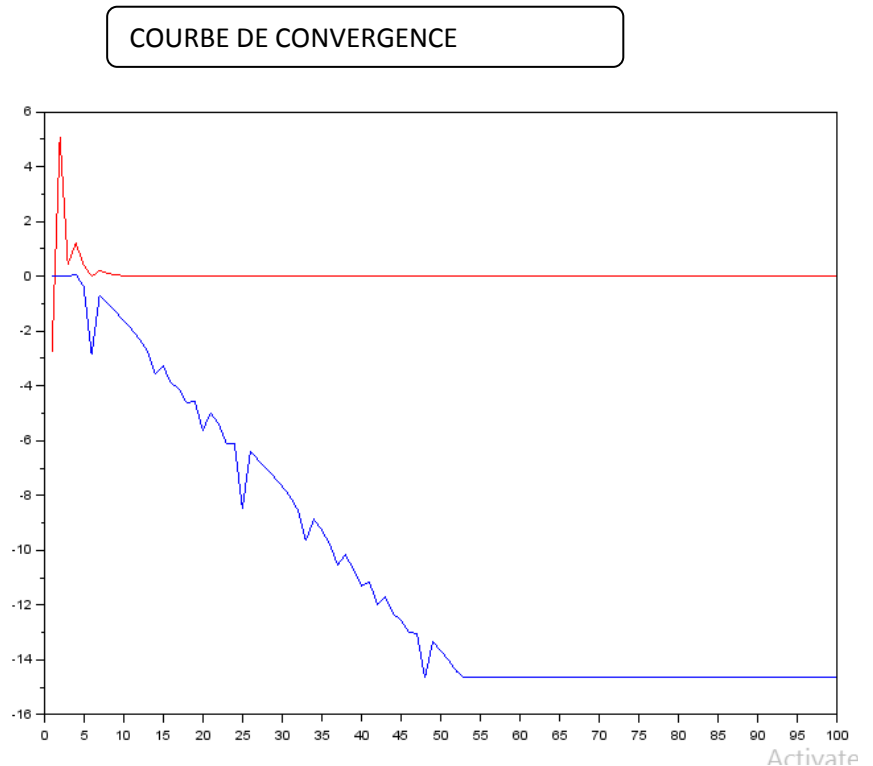
[6. 7] **6.4372982**

&CONVERGENCE DE DICHOTOMIE

```

a=6;
b=7;
x=(a+b)/2;
y(1)=f(a);
y(2)=f(b);
y(3)=f(x);
iter=3;
itermax=100;
while(iter<itermax)
    if(f(a)*f(x)<0)
        b=x;
    else
        a=x;
    end
    x=(a+b)/2;
    iter=iter+1;
    y(iter)=abs(f(x));
    z(iter)=log10(abs(f(x)));
end
plot(y,"r");
plot(z,"b");

```



La fonction $\text{abs}(f(x_n))$ tend vers 0 lorsque $x_n \rightarrow \alpha$

La fonction $\log_{10}(\text{abs}(f(x_n)))$ tend vers $-\infty$ lorsque $x_n \rightarrow \alpha$

Exercice 2:

Programmation des deux fonction f et f' pour utiliser la methode de newton

```

function [y]=f(x)
    y=x*tan(x)-1;
endfunction
function yprime=fprime(x)
    yprime=tan(x)+x/(cos(x))^2;
endfunction

```

1. la fonction `Newton(a,eps)` permet de calculer la valeur approchée à 10^{-5} près de la première solution positive de $f(x) = 0$:

```
function sol=Newton(a,eps)
```

```
    x=a;
```

```
    dif=1;
```

```
    while(dif > eps)
```

```
        xx=x-f(x)/fprime(x);
```

```
        dif=abs(xx-x);
```

```
        x=xx;
```

```
    end
```

```
    sol=x;
```

```
endfunction
```

```
// Q2
```

```
sol=Newton(0.5,0.1)
```

```
disp(sol)
```

La solution de $f(x)=0$

0.8710495

2. Programmation d'une fonction Scilab *vitesseNewton* pour tracer $f(x_n)$ en fonction du numéro d'itération n de l'algorithme :

```
x=1;
```

```
iter=1;
```

```
itermax=30;
```

```
y(1)=f(x);
```

```
while(iter<itermax)
```

```
    iter=iter+1 ;
```

```
    xx = x-f(x)/fprime(x);
```

```
    x=xx;
```

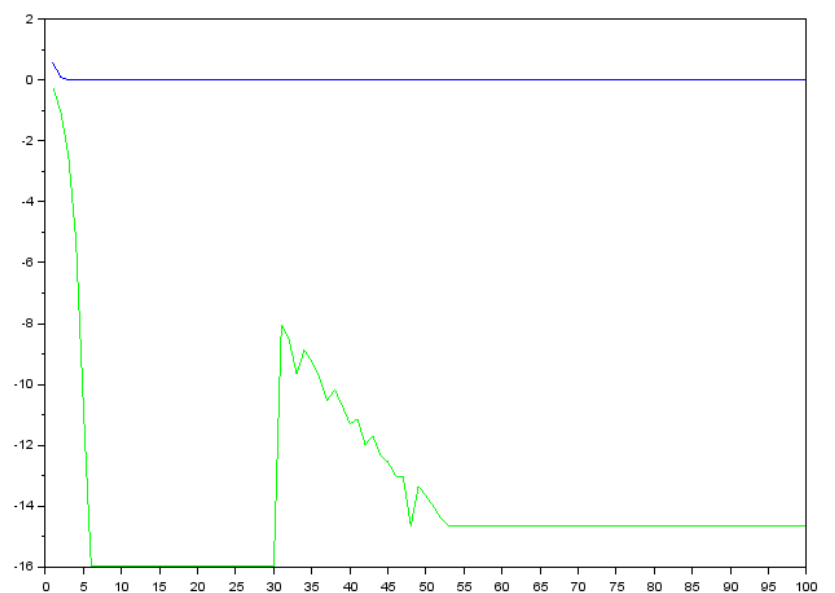
```
    y(iter)=f(x);
```

```
end
```

```
plot(y);
```

```
plot(log10(y),'g');
```

COURBE DE CONVERGENCE



TP3 : Interpolation polynomiale

Exercice 1

Le calcul des valeurs d'un polynôme $p(x)$ peut s'effectuer de deux manières :

La méthode classique (méthode de la somme des puissances)

Méthode de Horner

$P(x) = x(x(x(\dots(a_n) + a_2) + a_1) + a_0)$ cette écriture diminue le nombre d'opérations à faire pour évaluer $p(x)$ diminue la complexité

L'algorithme de Horner :

```
function y=horner(p, x)
    N=length(p)-1;
    y=p(N+1);
    for k=1:N
        y=y*x+p(N+1-k);
    end
endfunction
```

```
//application polynôme de d*2
//p(x) = 5x^2 - 3x + 1
p=[5 -3 1]
x=1;
y=horner(p,x);
disp(y);
```

La valeur de y est

P(1)=3

la méthode d'inversion de la matrice de VanderMonde

soit $n+1$ points d'interpolation définie comme vecteur ligne $X = (x_0, x_1, x_2, x_3, \dots, x_n)$;
et $Y = (y_0, y_1, y_2, y_3, \dots, y_n)$; les valeurs de $f(x_i)$

la fonction scilab interpolevdm :

```
function p=interpVDM(x, y)
```

```
    N=length(x)-1
    for i=1:N+1
        for j=1:N+1
            v(i,j)=x(i)^(j-1);
        end
    end
    p=inv(v)*y';
endfunction
x=[1 2 3]
y=[-1 1 0]
p=interpVDM(x,y);
disp(p);
```

Le résultat de l'exécution

```
-->exec('C:\Use
- 6.
  6.5
- 1.5
-->
```

Ca donne **$p(x) = -6x^2 + 6.5x - 1.5$**

TP4 : Intégration numérique

1. En cherchant à calculer numériquement l'intégrale $\int_a^b f(t)dt$

On a pour les points équidistants $x_k = a + k \frac{b-a}{n}, k = 0, 1, \dots, n$ la formule des rectangles composites s'écrit :

$$\int_a^b f(t)dt \approx \frac{b-a}{n} \sum_{k=0}^{n-1} f\left(a + k \frac{b-a}{n}\right)$$

1.1 la fonction intrectangles

```
function INT=intrectangles(a, b, n)
h=(b-a)/n;
s=0;
for k=1:n
    s=s+h*f(a+k*h);
end
INT=s;
endfunction
```

Cette valeur est la valeur approchée de l'intégrale de la fonction f entre 0 et 3

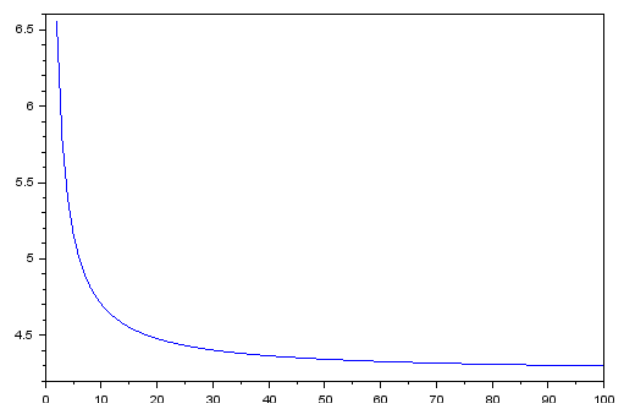
2- en définissant la fonction f(x)

```
function y=f(x)
y=x*sin(sqrt(x));
endfunction
INT=intrectangles(0,3,100);
disp(INT);
```

```
-->exec('C:\U
4.2980832
```

programme scilab pour tracer l'erreur

```
a=0;
b=3;
nmax=100;
nmin=2;
for n=nmin:nmax
    errRec(n-nmin+1)=intrectangles(a,b,n);
end
x=[nmin:nmax];
plot(x,errRec);
```



Le calcul analytique donne : $\int_0^3 x \sin(\sqrt{x}) dx = 6(\sin(\sqrt{3}) + \sqrt{3} \cos(\sqrt{3})) \approx 4,25$

La valeur exacte est loin de la valeur approchée d'où on propose d'augmenter le n pour d'approcher de la valeur exacte. On considère n=100 :